Now in SpringBoot to achive Dependency Inversion Principle?

Through Dependency Injection

What is Dependency Injection?

- Using Dependency Injection, we can makes our class independent of its dependencies.

- It helps to remove the dependency on Concrete implementation and inject the dependencies from external source.

loose couple

```
@Component
Public class User{
    @Autowired
    Oreder order
}
```

```
@Component
Public class Order{

}
```

@Autowired first look for a bean of required type.
-> If bean found, Spring will inject it.

Different ways of injection and which one is better?

1. Field Injection
2. Setter Injection
3. Constructor Injection

Concept && Coding

Field Injection :-

- Dependency is set into the fields of the class directly.
- Spring uses reflection, it iterates over the fields and resolve the dependency.

Advantages :-

- Very simple and easy to use.

Disadvantage

1 - Can not be use with Immutable fields.

```
@Component
Public class User {

  @Autowired
  Public final Order order;

  Public User() {
      S.o.p("Usr initialized");
  }
}
```

2 - Chanches a Null pointer Exception

```
@Component
Public class User {
  @Autowired
  Public Order order;

  Public User() {
      S.o.p("User initialized");
  }

  Public void process() {
    order.process();
  }
```

```
User userobj = new User();
UserObj.process();
```

Concept && Coding

- During Unit Testing. Setting Mock dependency into the field becomes difficult.


```
@Component
public class User {
  @Autowired
  private Order order;

  Public User( ){
    S.o.p("User initialized")
  }

  Public void process(){
    order.process();
  }
}
```

```
Class UserTest {
  Private Order orderMockObj;
  Private User user;

  @BeforeEach
  Public void setup(){
    orderMockObj = Mokito.mock(Order.class)
    this.user = new User();
  }
}
```

```
Class UserTest {

  @Mock
  private Order orderMockObj;

  @Inject Mocks
    privat User user;

    @BeforeEach
      Public void Setup(){
        MockitoAnnotations.initMocks(this);
      }
  }
```

Concept && Coding

Setter Injection

Dependency is set into the fields using the setter method. We have to annotate the method using @Autowired.

```
@Autowired
Public void setOrderDependency(Order order) {
    this.order = order;
}
```

Advantages :-

Dependency can be changed any time after the object creation (as object can not be marked as final.

Ease of testing, as we can pass mock object in dependency easily.

Disadvantage :-

Field Can not be marked as final (we can not make immutable).

Concept && Coding

Constructor Injection :-

- Dependency get resolved at the time of initialization of the Object itself.

- Its recommended to use

```
@Component
Public class User{

    Order order;

  @Autowire
  Public User(Order order){
        this.order = order;
      S.O.P("User initialized");
  }
}
```

```
@Component
@Lazy
Public class Order{

    Public Order(){
      S.O.P("order initialized
    }
}
```

* When only 1 constructor is present, then using @Autowired on constructor is not mandatory. (from spring version 4.3)
* When more than 1 constructor is present, then using @Autowired on constructor is mandatory.

Why Constructor Injection is Recommended (Advantages):

1:- All mandatory dependencies are created at the time of initialization itself. Marks 100% Sure that our object is fully initialized with mandatory dependency

(i) avoid NPE during runtime

(ii) Unnecessary null checks can avoid to

Concept && Coding

2:- We can create immutable object using Constructor injection. (8)

3:- Fail Fast :- If there is any missing dependency, it will fail during compilation itself, rather than failing during run Time.

4:- Unit testing is easy :-

```
@Component
Public class User{
  private Order order;

  @Autowired
  Public User(Order order){
      this.order = order;
      S.o.p("User initialized")
  }

  Public void process(){
      order.process;
  }
}
```

```
Class UserTest{
  Private Order orderMockobj;
  Private User user;

  @BeforeEach
  Public void setup(){
    this.orderMockobj = Mockito.Mock
                        (Order.cl
    this.user = new User(orderMock
                         obj)
  }
}
```

Concept && Coding

Common Issues when dealing with Dependency Injection

CIRCULAR DEPENDENCY

```
@Component
Public class Order{
  @Autowired
  Invoice invoice;

  Public Order(){
      S.O.P(" ");
  }
}
```

```
@Component
Public class Invoice{
  @Autowired
  Order order;

  Public Invoice(){
      S.O.P(" ");
  }
}
```

Solutions

1:- First and foremost, can we refactor the code and remove this cycle dependency.

For example :- Common code in which both are dependent, can be taken out to separate class. This way we can break the circular dependency.

2 :- Using @Lazy on @Autowired annotation.
Spring will create proxy bean instead of creating the bean instance immediately during application starup.

3 :- Using @PostConstruct.

Concept && Coding

Qualifier :- This is used to tell Spring exactly which bean to use by specifying its name or custom level. It helps when there are multiple bean, and you want to choose a specific one rather than using default.

@Primary :- This is used when there are multiple beans of the same type and you want one of them to be the default choice. If Spring does not know which bean to use, it will pick the one marked with @Primary automatically.

Concept && Coding